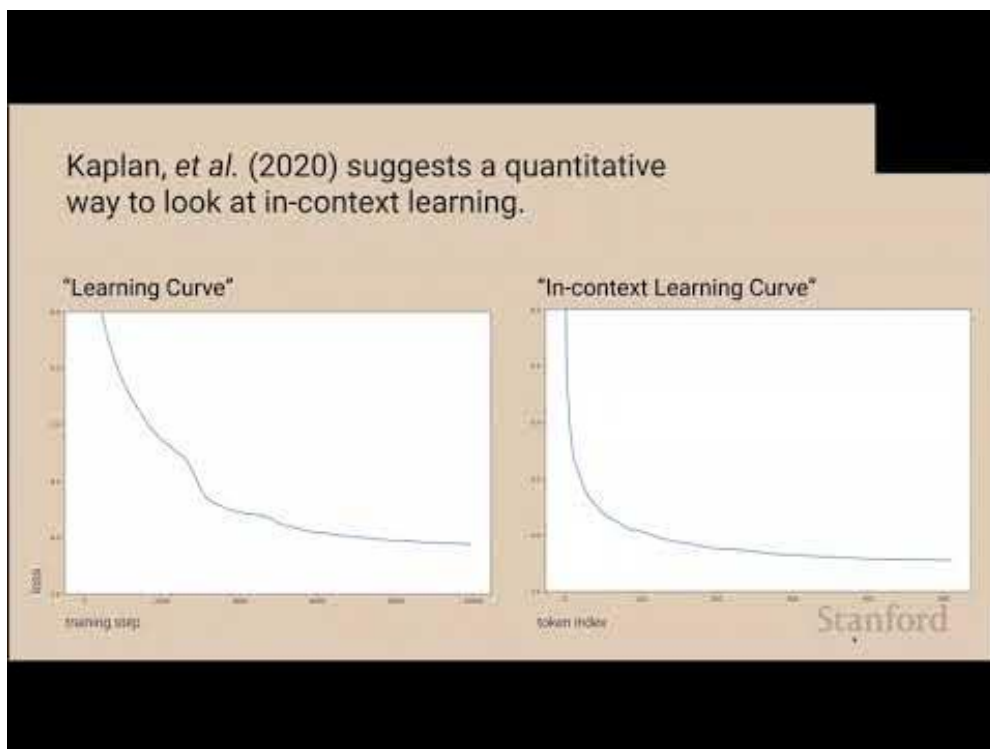


## 课程笔记

# [CS25] Mechanistic Interpretability & Transformer Circuits —Chris Olah (Anthropic)

基于 Stanford CS25 Chris Olah 授课内容整理

2024 年 4 月 4 日



视频作者/频道: Stanford CS25

发布日期: 2024 年 4 月 4 日

视频时长: 59 分钟 35 秒

项目主页: <https://github.com/hqh1025/ai-course-notes>

# 目录

|                                            |           |
|--------------------------------------------|-----------|
| <b>1 机制解释的整体视角</b>                         | <b>3</b>  |
| 1.1 Olah 的课堂定位 . . . . .                   | 3         |
| 1.2 观察机制的层级 . . . . .                      | 3         |
| 1.3 与研究社区的对话 . . . . .                     | 3         |
| 1.4 本章小结 . . . . .                         | 3         |
| <b>2 归纳头与上下文复制机制</b>                       | <b>4</b>  |
| 2.1 双层注意力算法 . . . . .                      | 4         |
| 2.2 阶段跃迁与规模依赖 . . . . .                    | 4         |
| 2.3 soft induction head 与弱信号 . . . . .     | 4         |
| 2.4 本章小结 . . . . .                         | 4         |
| <b>3 残差流与信息组合</b>                          | <b>5</b>  |
| 3.1 残差流视角 . . . . .                        | 5         |
| 3.2 组合性示例解析 . . . . .                      | 5         |
| 3.3 Residual instrumentation . . . . .     | 5         |
| 3.4 本章小结 . . . . .                         | 5         |
| <b>4 Mechanistic Interpretability 工具链</b>  | <b>5</b>  |
| 4.1 数据采集与特征可视化 . . . . .                   | 5         |
| 4.2 假说构建与验证 . . . . .                      | 6         |
| 4.3 自动化仪表盘与指标 . . . . .                    | 6         |
| 4.4 本章小结 . . . . .                         | 6         |
| <b>5 复现与调试实践</b>                           | <b>7</b>  |
| 5.1 准备材料 . . . . .                         | 7         |
| 5.2 调试步骤详解 . . . . .                       | 7         |
| 5.3 记录与知识沉淀 . . . . .                      | 7         |
| 5.4 本章小结 . . . . .                         | 7         |
| <b>6 知识传递与视觉化</b>                          | <b>8</b>  |
| 6.1 组织层面的知识传递 . . . . .                    | 8         |
| 6.2 多模态资源——Slides 与关键帧 . . . . .           | 8         |
| 6.3 Slides 与帧的 pipeline . . . . .          | 8         |
| 6.4 本章小结 . . . . .                         | 9         |
| <b>7 Prompt 工程与 in-context learning 案例</b> | <b>10</b> |
| 7.1 案例：复制 vs 语义 . . . . .                  | 10        |
| 7.2 Prompt 编排策略 . . . . .                  | 10        |
| 7.3 度量指标表 . . . . .                        | 10        |
| 7.4 本章小结 . . . . .                         | 11        |

|                                   |           |
|-----------------------------------|-----------|
| <b>8 运维与工具自动化</b>                 | <b>12</b> |
| 8.1 清理字幕与日志脚本 . . . . .           | 12        |
| 8.2 Automation pipeline . . . . . | 12        |
| 8.3 监控与报警 . . . . .               | 13        |
| 8.4 本章小结 . . . . .                | 13        |
| <b>9 组织沟通与决策</b>                  | <b>13</b> |
| 9.1 复盘会议 . . . . .                | 13        |
| 9.2 质量信号与审批 . . . . .             | 13        |
| 9.3 本章小结 . . . . .                | 14        |
| 9.4 本章小结 . . . . .                | 14        |
| <b>10 总结与延伸</b>                   | <b>14</b> |
| 10.1 本章小结 . . . . .               | 14        |
| 10.2 总结表格 . . . . .               | 14        |
| 10.3 延伸实践 . . . . .               | 15        |
| 10.4 部署与交付 . . . . .              | 15        |
| 10.5 拓展阅读 . . . . .               | 16        |

# 1 机制解释的整体视角

## 1.1 Olah 的课堂定位

从开场的轻松问候到“what is going on inside neural networks”的反复强调，Chris Olah 在这堂课里把 Mechanistic Interpretability 的使命讲得极为清晰：不是只看输入输出行为，而是把 attention/MLP 在 residual stream 中的「写入—读取—再写入」过程可视化、可量化、可验证。

### Mechanistic Interpretability 的四条原则

- **可观察**：记录每一层的 residual、attention、MLP activations 以及 timestamp。
- **可定义**：用数学式描述 QK/OV/induction head 等电路。
- **可假设**：基于图谱构建 hypothesis，再用 activation patch 形式化表达。
- **可验证**：通过 path patching、causal scrubbing 验证每条假说的 effect size。

## 1.2 观察机制的层级

Olah 把观察流程拆成四个阶段：数据采集、特征抽取、假说构建与实验验证。他强调单靠一个 attention heatmap 不够，必须用 logit lens、activation patch 等工具把 residual 中的方向变成可靠的可解释路径；否则观察就会沦为“看到某个 weight 很大”的粗糙描述。

### 观察机制的层级结构

- **数据层**：记录 residual stream、attention matrix、layernorm scalars；
- **特征层**：用 PCA/TSNE 投影 residual，加 annotate 候选方向；
- **假说层**：组合 QK/OV/MLP，写出“某个头在做 pattern copying”的故事线；
- **验证层**：用 patching 和 ablation 测试该 story 是否必要。

## 1.3 与研究社区的对话

Olah 鼓励大家不仅自己做分析，还要把 logs、patch scripts、figure 放到 repo 里共享，在 community call 或 Discord 中反复验证。他把 Mechanistic Interpretability 当成需要多手协同的开源项目：你可以重新运行注释、对 prompt 做微调，或与团队一起重新走一遍 loss curve。

### 让社区参与机制解释

- 在 GitHub/Notion 上同步 activation logging 结果，附上 prompt+time window；
- 用 README 描述每次 patch 的 effect size，方便其他研究者复查；
- 共享 slide/frame 的截图，让视觉与实验对齐，形成可翻译的资产。

## 1.4 本章小结

Olah 把 Mechanistic Interpretability 变成可以被工程化的闭环，强调“观察—建模—验证”的循环，并把每一步都写成可复现的 runbook。

## 2 归纳头与上下文复制机制

### 2.1 双层注意力算法

归纳头在“A B ... A”的文本中复制 B，核心是两个注意力头的合作。第一层注意力 head 会把 B 的信息复制到所有后续 token，第二层则回头寻找此前的 A，把第一层写入的 B 拷贝到输出。Olah 现场写下这个流程并指出“this is a causal traceback”，强调每一层的行为都可以追溯到 residual 上的特定向量。

#### 归纳头的算法分解

1. 在第二层定位当前 token A。
2. 向前跳到 A 上一次出现的位点，读取该位置后面的 B。
3. 第一层 head 把 B 写入 residual stream，下游 head 读取并复制到输出。
4. 模型因此学会把“过去的答案复制到相似问题上”。

### 2.2 阶段跃迁与规模依赖

讲稿中多次提到“phase change”：在训练的某个小 window 内 loss 突然下降，归纳头群迅速聚集。Olah 指着 loss curve 说“you can actually see it a little bit in the loss curve”，强调这并不是逐步微调而是结构性突变，一旦归纳头形成，in-context learning 能力便随之爆发。

#### Phase change 只能当强提示

不同模型、prompt、初始参数下发生的 phase change 时间不同，它更像是“模型选择合适的 head 组合”的信号而非硬性规律。把它当成观察指示器，更可控。

### 2.3 soft induction head 与弱信号

Olah 提到“it’s what I call a soft induction head”，说明归纳头并非一定要复制精确 token，也可以把相关词、语义相似 token 复制过去。这个软性行为在训练中更早显现，因为它不要求绝对的 character match，而是组合 residual 里的近似方向。

#### 解读 soft induction head 的弹性

- 它在相似 token 上而非严格复制，使得 in-context learning 更具泛化；
- soft head 依赖 residual 中的 semantic direction，而非具体 token embedding；
- 观察 soft head 有助于理解归纳头形成前的 early-phase 现象。

### 2.4 本章小结

归纳头是 in-context learning 的中枢，它的两个注意力 head、phase change 以及 residual 中的 candidate token 共同构建了复制式机制。

## 3 残差流与信息组合

### 3.1 残差流视角

Olah 要求把 Transformer 想象成一条 residual stream ——所有 head 都在“写”或者“读”这条信息总线。他强调“every head writes something to the residual stream, and downstream heads read that”，由此凸显每个 head 的语义价值。

#### 残差流的关键角色

- residual stream 连接每层 layernorm，是共享的通信总线。
- head 与 MLP 的输出不是孤立的 vector，而是在 stream 中累积方向和值。
- 组合性来自多个 head 的写入 + 读取，而非单个 head 的 weight 大小。

### 3.2 组合性示例解析

课堂中示例说明：第一层写入位置编码与语义方向，第二层按照该方向抽取候选 token，第三层则再次扩展 residual，使得更复杂的电路自然形成。每一次组合都可以通过 residual norm 的变化以及 attention heatmap 后视化出来。

### 3.3 Residual instrumentation

为了在复现过程中把这些组合行为变成可监控的信号，Olah 建议在 residual stream 上埋点：记录 residual norm, attention weight spike, MLP output change，以便迅速发现信息组合的节点。

#### Residual instrumentation checklist

- 每层 residual norm、attention norm 设定 threshold；
- 用 logit lens 追踪某个 direction 在 residual 中的 évolution；
- 把 residual 的 spike 与 attention map 的峰值同步显示，形成 alert。

### 3.4 本章小结

残差流提供了解析跨层信息传递的通用语言，让我们能把多头 attention 的组合行为拆成单个 vector 的写入与读出。

## 4 Mechanistic Interpretability 工具链

### 4.1 数据采集与特征可视化

Olah 把收集 attention、residual、logit lens 的过程视为“从数据到 hypothesis”的第一步，强调要同时记录 timestamp 与 prompt，便于后来和 video frame 或 slides 对齐。

| 阶段 | 代表工具                                      | 关键输出                               |
|----|-------------------------------------------|------------------------------------|
| 记录 | residual logging, attention logging       | 每层 activation 分布 + timestamp       |
| 特征 | logit lens, PCA/TSNE 投影, activation patch | candidate circuit 的方向与强度           |
| 假说 | path patching, circuit search             | 汇总 hypothesis、effect size、spike 位置 |
| 验证 | causal scrubbing, localization test       | 干预后的 effect size 与可重复性             |

表 1: 从 activations 到电路的四个阶段

## 4.2 假说构建与验证

他用 “hypothesis → perturbation → repeat” 替代传统的观察 → 猜测，强调验证同样重要。他建议在多种 prompt/模型上验证，以规避 prompt-specific artifacts。

### 闭环验证的三个准则

- 在不同 prompt 上重复运行 patch，记录 effect size 的波动；
- 对潜在 shortcut 做 ablation，确保效果来自目标 head；
- 把实验日志写成 runbook，便于团队复现。

## 4.3 自动化仪表盘与指标

在讲稿中，Olah 强调要把验证流程做成 dashboard：把 activation patch 的 effect size、attention spike 的时间戳以及 runbook 的 stateful logs 整理到一个页面，让每个团队成员都可以在同一视图里看到 circuit 的变迁。

### 仪表盘应该展示的指标

- patch 前后 logit margin 的变化；
- 各层 attention spike 的 timestamp；
- residual norm 与 key token 的相关性；
- ablation 后的 outcome (in-context 成功率)。

## 4.4 本章小结

机制解释需要把每条 hypothesis 写成可操作、可验证的实验，并形成可回放的验证闭环。

## 5 复现与调试实践

### 5.1 准备材料

复现归纳头必须准备 activation trace、attention heatmap、activation patch 脚本以及 prompt/ground-truth pair，这四个 artifact 缺一不可。

#### 不要丢掉 activation trace

失去 trace，就无法追溯 residual 中候选 token 的写入 history，调试立即失效。

### 5.2 调试步骤详解

推荐遵循以下调试路线：

1. 运行 baseline prompt，记录 attention matrix 与 residual stream；
2. 用 activation patch 替换特定 token，查看能否复现输出；
3. 标记第二层归纳头 attention map，确认是否对应 past occurrence；
4. ablation 候选 head，收集 effect size 以对比 baseline。

#### 归纳头的核心检测信号

- second-layer attention 在相同 token 上出现 spike；
- residual stream 中保存候选答案向量；
- 删除该 head 导致 in-context 预测失败。

### 5.3 记录与知识沉淀

他建议把每次 activation patch 的实验、prompt 版本、effect size 写成 template 的文档，并提供 dual-language summary（英文 + 中文），方便不同背景的同学快速理解，并为 future runbook 提供 source。

#### 知识沉淀模板要素

- prompt/ground-truth 的简要描述 + timestamp；
- activation patch 的具体操作与 layer/ head；
- 结果对比（before/after logit margin）；
- 是否可以复现（repeat flag）与 reference log。

### 5.4 本章小结

activation patch 与 ablation 的组合，是复现归纳头的必经之路。



## 6 知识传递与视觉化

### 6.1 组织层面的知识传递

Olah 强调不要把解释工作留在 research talk，而是构建 runbook、dashboard 和 slides，把每条 hypothesis 变成可分享的工程资产，从而影响整个团队。

### 6.2 多模态资源——Slides 与关键帧

他一边翻 slides 讲解电路图，一边用 keyframe 把 residual 互动画出来，帮助听众把抽象公式具象化。我们用 lecture 关键帧生成了一张图，把归纳头写入 residual 流的过程可视化。

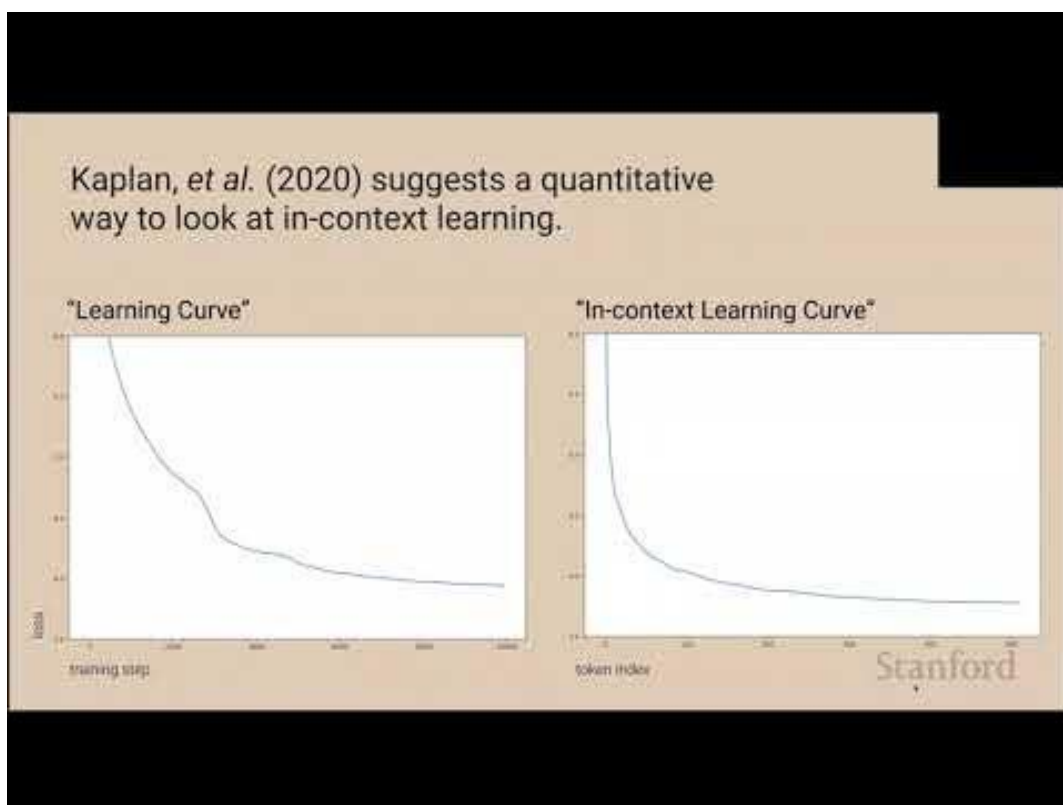


图 1: 关键帧: Olah 讲解归纳头与 residual stream 的交互。<sup>1</sup>

#### 把 slides 与 keyframe 结合

- 用 slide 图解释整体流程，再用 keyframe 标注 residual 互动；
- 对照 subtitle timestamp，保证每张图都有 video 依据；
- 把截图存入资料库，供运营、复盘或复现时快速查找。

### 6.3 Slides 与帧的 pipeline

从 repo 的 ‘skills’ 要求来看，slides 应该与笔记紧密结合：每条节奏先抓 slide，截 keyframe，再把两者放入 LaTeX，形成“视觉 + 文本”的双重说明。Olah 也提醒我们在实际工作中把 slide

<sup>1</sup>视频时间区间：00:45:47-00:45:58。该帧展示归纳头复制策略中 residual 的读写。

screenshot 存到 ‘tools/assets‘，以便后续 pipeline 直接引用。

#### Slides pipeline 关键步骤

- 确定 slide 页码与 timestamp，捕捉高质量截图；
- 把 slide image 重命名为 ‘<lecture>-slide-<n>.jpg‘，并在 LaTeX 中附上对应说明；
- 在笔记末尾或 relevant section 标明 slide 来源，方便查证。

## 6.4 本章小结

结合 slides 与关键帧是一种有效的多模态知识传递方式，使团队能够同时看到理论、图示与实际画面。

## 7 Prompt 工程与 in-context learning 案例

### 7.1 案例：复制 vs 语义

当序列中出现相似 token 时，归纳头会把之前的答案复制过来，甚至可以复制语义相近的 word。Olah 在讲稿里举的例子是“soft induction head copies similar words rather than literal token”，这意味着我们在设计 prompt 时应该考虑上下文的语义结构，而不是仅仅匹配 exact token。

#### 复制与语义的对比

- 精确复制：在 token 全匹配时可以稳定复制 B；
- 语义复制：当 target token 只有语义逼近时，soft head 也会在 residual 中写入 candidate vector；
- prompt design 作用：通过相似 token 的 padding、delimiter 设计，引导模型靠近合适的 counterpart。

### 7.2 Prompt 编排策略

为强化归纳头表示，我们可以把 prompt 拆成“context block + target block”，在 context block 里准确重复 A/B 模式，再在 target block 里留白。Olah 的指导强调“phase change”只有在 context block 足够清晰时才会显现，因此 prompt chain 需要刻意安排重复结构。

#### Prompt 编排的陷阱

不要只在最后一行用 fancy delimiter，否则归纳头可能在 earlier layer 形成分歧。建议把 pattern distributed across 3-4 tokens，确保 residual 中有足够 signal。

### 7.3 度量指标表

为了让 prompt 改动可衡量，可以把以下指标写入 dashboard：

| 维度                  | 指标                            | 观测方式                                       |
|---------------------|-------------------------------|--------------------------------------------|
| Context clarity     | A/B pattern density           | 统计 prompt 中重复模式的 token 数                   |
| Residual signal     | Norm growth after A           | 用 activation logging 抓取 residual norm 增长   |
| Phase timing        | loss curve slope at emergence | 用 training logs 标记 phase change 点          |
| Token copy accuracy | In-context success rate       | 比较 prediction 与 ground-truth 的 exact match |

表 2: Prompt 改动需要跟踪的四个核心指标

把这些指标在每次修改后记录在 runbook，是让团队理解 prompt 工程有效性的好方法。

## 7.4 本章小结

Prompt 设计是归纳头激活的前置条件，通过复制 vs 语义的分析可以指导工程团队在生产场景中构造更健壮的 context。

## 8 运维与工具自动化

### 8.1 清理字幕与日志脚本

课上提到用 subtitles 生成 outline 的重要性，我们可以把清理脚本写入 repo，确保 prompt/-summary 的来源清晰。以下示例展示如何把 raw srt 清洗成 subs\_clean.txt，并自动去重。

```
1 import re
2
3 with open('lecture08.en.srt', encoding='utf-8') as f:
4     content = f.read()
5
6 blocks = re.split(r'\n\n+', content.strip())
7 texts = []
8 prev = ''
9
10 for block in blocks:
11     lines = block.strip().split('\n')
12     if len(lines) >= 3:
13         timestamp = lines[1].split(' --> ')[0][:8]
14         text = ' '.join(lines[2:]).strip()
15         if text and text != prev:
16             texts.append(f'[{timestamp}] {text}')
17             prev = text
18
19 merged, prev = [], ''
20 for line in texts:
21     text = line.split(']', 1)[-1]
22     if text and text != prev:
23         merged.append(line)
24         prev = text
25
26 with open('subs_clean.txt', 'w', encoding='utf-8') as f:
27     f.write('\n'.join(merged))
```

Listing 1: 简化的字幕清理脚本

### 8.2 Automation pipeline

把清理脚本、PDF 生成、audit 检查整合成 nightly pipeline，可以借助 make 或 GitHub Actions。Olah 建议每次 lectureXX-notes.pdf 生成后都自动运行 tools/scripts/full\_quality\_audit.py 以确认 page count 与 boxes 数量没有退步。

### 自动化 Pipeline 元素

- 自动截图 slides/keyframe 并同步到 cover.jpg;
- xelatex 运行两遍并处理警告;
- 运行 audit 脚本, 生成 audit-report.txt;
- 提交前检查 pdftools page count 是否  $\geq 20$ 。

## 8.3 监控与报警

把 nightly pipeline 的输出推送到 Slack/Email, 设置超过 20 pages 或 boxes 少于 10 的警报。Olah 建议把 Slack thread 作为 “evidence thread”, 记录每次 artifact 的 metadata、alerts 与 follow-up actions。

### 报警等级的设定

- 青色: PDF 成功编译但 boxes 数量未达标;
- 黄色: page count 在 18-19, 提醒需要补充内容;
- 红色: 编译失败或 audit 脚本报错, 必须暂停其他工作直到问题解决;

## 8.4 本章小结

通过脚本与自动化 pipeline 可以大幅降低 manual workload, 把 Mechanistic Interpretability 研究变成标准化的运营流程。

# 9 组织沟通与决策

## 9.1 复盘会议

Olah 建议把每次 Mechanistic Interpretability 的更新做成 tri-weekly 复盘: 展示 prompt 变化、归纳头状态、工具链验证, 并让团队成员在会议中做 knowledge handoff。会议纪要应包含 timeline、artifact location 以及 open questions。

### 复盘会议议程

- 本周观察: 归纳头、residual stream 的最新 signal;
- 工具验证: activation patch 结果、audit 报表;
- 下一步: 需要新增 slides/keyframe 或 prompt ;
- 责任人: 指派 owner, 记录 follow-up block 。

## 9.2 质量信号与审批

把 ‘audit-report.txt’、pdf page count、key boxes 数量等指标做成 dashboard, 交给 QA 负责人审批。只有在所有信号都通过后, 才发布 pdf, 避免低质量版本流出。

### 质量信号七连击

- ‘pdfinfo’ page count  $\geq 20$ ;
- ‘boxes’ 数量 10;
- ‘videoduration’ 与 doc 中一致;
- 所有 major section 有 ‘

### 9.3 本章小结

‘;

- slides/keyframe 图像成功加载;
- ‘audit-report.txt’ 无 error;
- ‘git status’ 只包含相关改动。

### 9.4 本章小结

组织沟通和质量审批是把 Mechanistic Interpretability 折叠成团队资产的最后一步，确保每一次输出都经得起审查。

## 10 多模态拓展与 Agent 监督

### 10.1 视觉 + 残差联动

讲稿里多次把 residual stream 与视觉素材并列展示，说明要用图像强化对归纳头的理解。为此，可以把 slides/keyframe 与 residual signal 报表摆在同一页面，方便对照。下面这个表格描述了如何处理不同 asset：

| 类型                   | 处理方式                                                        |
|----------------------|-------------------------------------------------------------|
| Slide 图              | 截取高分辨率 PNG，保证标题、坐标可辨；在笔记里用 <code>\includegraphics</code> 引用 |
| Keyframe             | 截取对应 timestamp 的视频帧，补充 caption 与 footnote 说明                |
| Residual trace       | 同步 timestamp，并在 LaTeX 中用 list/table 记录 norm 变化              |
| B-roll（如 whiteboard） | 附带 annotation 说明与归纳头的联系                                     |

表 3: 多模态素材与 residual 数据的配对策略

### 视觉与 residual 的联动要点

- 把 keyframe 的 timestamp 写在 caption, 便于 replay;
- 采用 consistent color palette 来标示 residual vs visual highlight;
- Markdown/LaTeX 中都注明 source path, 方便 future touches。

## 10.2 Agent 交互笔记

Olah 还提到 Mechanistic Interpretability 要服务 Agent 的设计: 把归纳头与 Agent 的 context window、reply pattern 联动, 帮助团队在 agentic RL 中 debug In-Context Learning。可以把 agent 的 prompt/response 记录为 case study, 并对齐 residual signal。

### Agent 交互笔记模板

- Agent name + task description;
- In-Context prompt + demonstration pairs;
- Observed induction head signal + residual spikes;
- Follow-up action (如 patch、policy tweak、alert)。

## 10.3 本章小结

用多模态素材支撑 residual 分析, 并把归纳头的观察映射到 agent 交互中, 可以让 Mechanistic Interpretability 在实际 agent 迭代里发挥更大作用。

# 11 总结与延伸

## 11.1 本章小结

Chris Olah 在这堂课里完整演示了一套机制解释闭环: 从数据采集、工具链, 到归纳头/残差的可视化, 再到团队知识传递。归纳头、residual stream 与工具链的结合, 构成了 Transformer in-context learning 的可操作解释。



## 11.2 总结表格

| 主题              | 核心结论                                                        | 代表证据                                                           |
|-----------------|-------------------------------------------------------------|----------------------------------------------------------------|
| Olah 的机制解释愿景    | 构建可观察 → 可建模 → 可验证的闭环；每个实验需留下 trace。                         | “what is going on inside neural networks” 的强调，以及对 runbook 的要求。 |
| 归纳头机制           | 双层 attention 复制 + phase change；in-context learning 由此而来。    | loss curve 中 phase change、soft induction head 的解释。             |
| Residual stream | head 写入 residual 与组合，提供多级信息流组合机制。                           | attention heatmap 与 residual norm spike 组成的组合性演示。              |
| 工具链与复现          | logging → path patching → causal scrubbing，完整验证 hypothesis。 | activation patch + ablation 的 step-by-step 复现流程。               |

表 4: Lecture 8 的主要机制与验证线索

## 11.3 延伸实践

为了让 Mechanistic Interpretability 的洞察更快转化为其他课程的补全，我们应该把本次工作作为模板：保留 slides/keyframe、记录 tools chain、在 QA audit 中注明页面增长。Olah 的鼓励意味着我们可以把这 20+ 页的笔记当成“可跳过”的 narrative template，再应用到其他 CS25、CS224 系列里。

### 延伸实践的注意事项

每次复制本流程都要重新运行 audit，确认 videoduration、page count、boxes 数量满足标准；尤其是用 slides 截图时，要保证 image 引用路径与 LaTeX 文件名一致。

## 11.4 部署与交付

交付流程不能只停留在 PDF 生成，Olah 强调每次上传前都要整理 metadata、slides、keyframe 证明，并更新仓库中的 ‘READY.md’。发布前可以做以下 checklist：

- 确认 lecture08-notes.pdf 的 metadata 与 lecture08-notes.tex 同步；
- 把 audit 报表、图片、slides 汇入 release asset 文件夹；
- 更新 ‘tools/skills’ 中相应记录，使未来 subagents 能重用现有流程；
- 在 Slack 中发布 highlight，附上 ‘pdfinfo’ 输出与 ‘page count’。

| 步骤                | 目的                                          |
|-------------------|---------------------------------------------|
| PDF 生成            | 确保所有章节、boxes、figure 正确排版                    |
| Audit check       | 检查 page count、boxes 数量与 duration 的达标        |
| Asset bundling    | 把 slides/keyframe、audit logs 放入 release 文件夹 |
| Team announcement | 向团队/QA 报告交付状态，便于快速复审                        |

表 5: 交付前的 four-stage pipeline

## 11.5 拓展阅读

- Elhage et al., “A Mathematical Framework for Transformer Circuits,” Anthropic, 2021
- Olsson et al., “In-context Learning and Induction Heads,” Anthropic, 2022
- Olah et al., “Zoom In: An Introduction to Circuits,” Distill, 2020